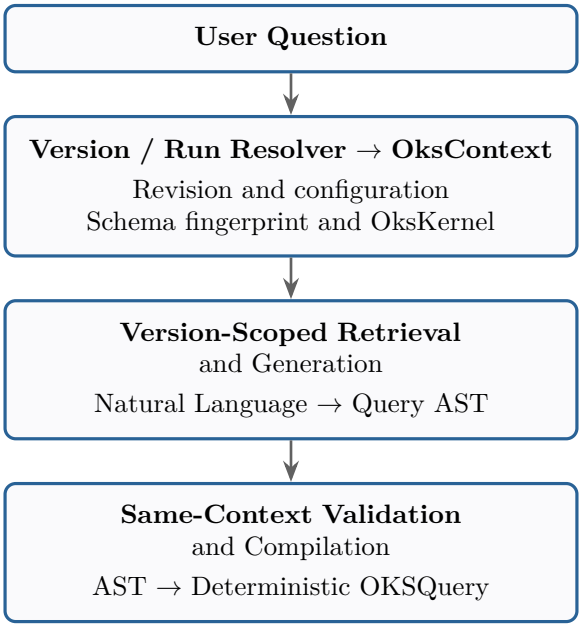
NL → OKSQuery
System Architecture

A Version-Scoped Deterministic Pipeline for Natural Language to Schema Query Translation



Architecture Specification Document

August 24, 2026

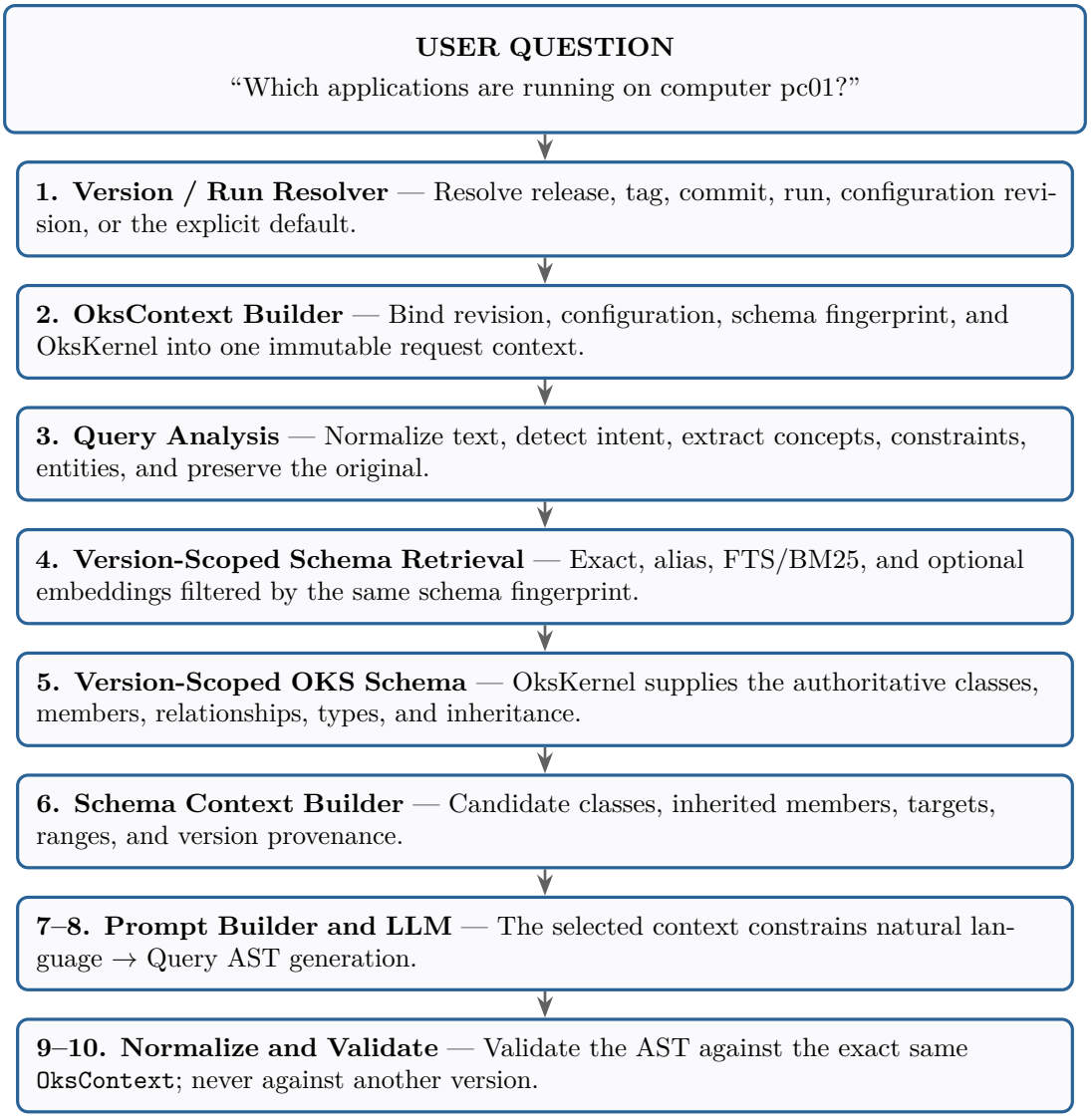
Contents

1 Complete Architecture

The system translates natural-language questions into validated, executable OKSQuery statements through a version-scoped 12-module pipeline. Every request first resolves an immutable `OksContext`; all later retrieval, schema access, generation, validation, repair, compilation, and execution operations use that same context.

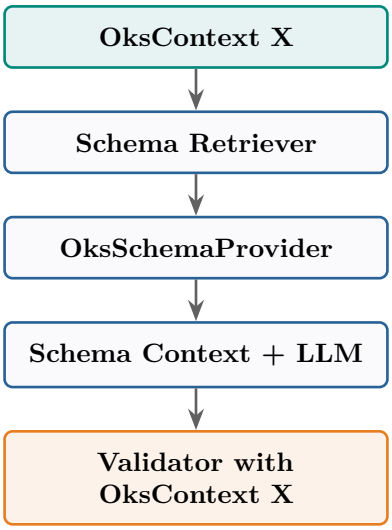
1.1 High-Level Data Flow

The user question flows through the pipeline as follows:



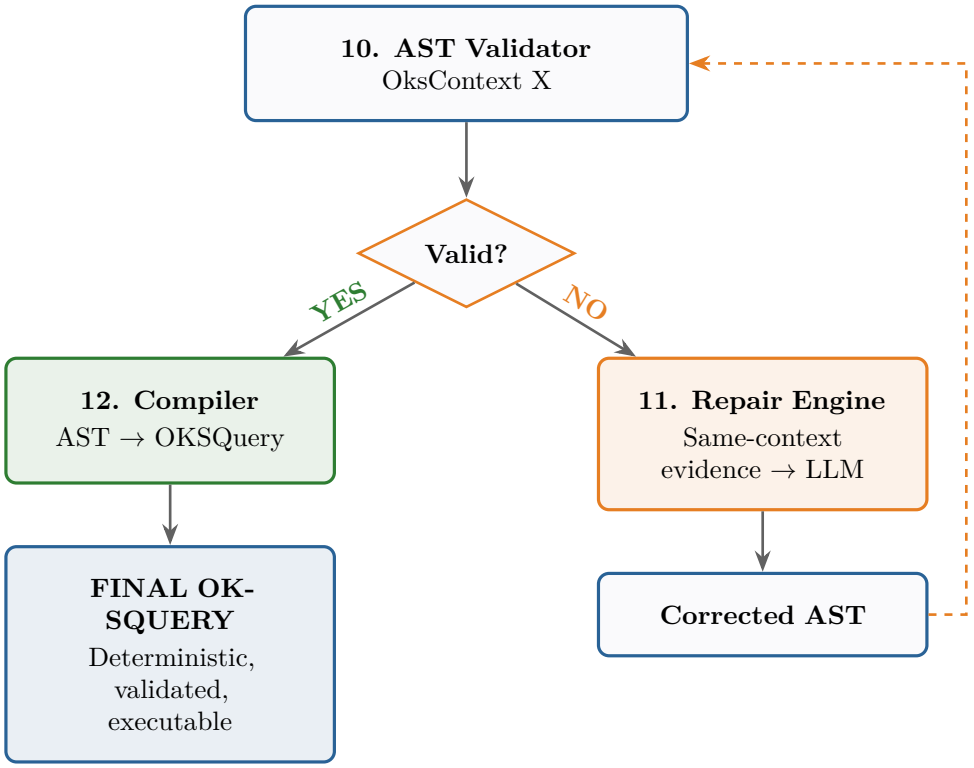
1.2 Schema-Context Invariant

Schema-context invariant. Every retrieval, schema expansion, LLM context, validation, repair, compilation, and execution operation for a query must use the same resolved `OksContext`. No schema information from another release, Git revision, configuration, or run may be used unless the request explicitly asks for cross-version comparison.



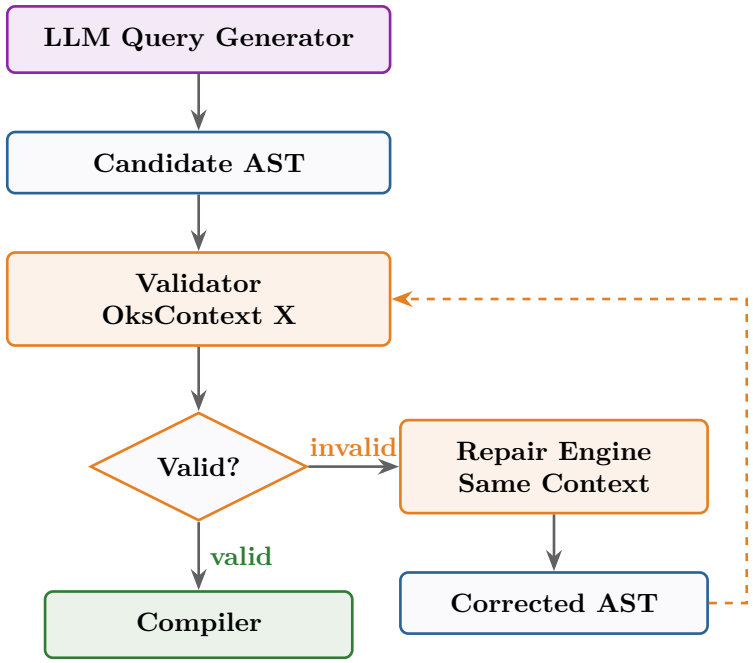
1.3 Validation Branch

After the AST Validator, the pipeline branches based on validation result:



1.4 Repair Loop

If validation fails, the Repair Engine engages the LLM with focused feedback:



Hard limit: `max_repair_attempts = 2`

2 Module 1 — Version / Run Resolver

The Version / Run Resolver determines exactly which OKS configuration and schema apply to the request. It runs before schema retrieval because retrieval results are meaningful only within a resolved version and configuration.

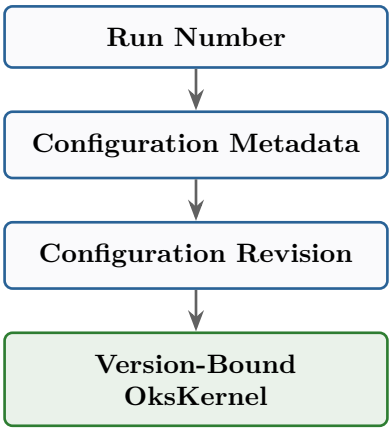
2.1 Accepted Selectors

The resolver accepts one of the following selectors, either from the request metadata or explicitly from the user's question:

- TDAQ release;
- Git tag;
- Git commit;
- run number;
- configuration version or configuration revision;
- no selector, in which case the currently loaded/default configuration is used and recorded explicitly.

2.2 Run Resolution

A run number is not treated as a schema version. It is resolved through configuration metadata to the corresponding configuration revision and then to the version-bound `OksKernel`:



If the mapping cannot be resolved, the request fails explicitly. The system must not silently substitute the current schema.

3 Module 2 — OksContext Builder

The OksContext Builder creates the immutable context shared by every downstream component. The name `OksContext` is used because the object identifies both the schema and the loaded historical configuration.

```
class OksContext:
    release: str | None
    git_revision: str | None
    run_number: int | None
    configuration_revision: str | None
    schema_identifier: str
    schema_fingerprint: str
    kernel: OksKernel

class QueryRequest:
    oks_context: OksContext
    ast: QueryAST
```

The semantic AST describes *what* to query. The `OksContext` records *where* and *against which exact schema/configuration* the query is interpreted.

3.1 Schema Fingerprint

The context includes a stable schema fingerprint, for example:

```
<release>:<git-revision>:<schema-root-hash>
```

The release name alone is insufficient because different Git revisions can change schema files. The fingerprint is used in retrieval filters, caches, logs, validation diagnostics, and provenance.

3.2 Request Isolation

An `OksContext` is immutable for one request. If the user changes version during a conversation, the next request receives a new context. An explicit cross-version comparison creates two or more contexts and uses a separate comparison layer; ordinary query translation never mixes them.

4 Module 3 — Query Preprocessor

This is the first semantic-analysis layer after context resolution. Its job is to prepare the raw question for downstream retrieval without losing information or detaching it from the resolved `OksContext`.

4.1 Example Input / Output

Input:

```
Which applications are running on computer pc01?
```

Output:

```
{
  "original_query": "Which applications are running on computer pc01?
     ",
  "normalized": "applications running on computer pc01",
  "concepts": ["application", "running on", "computer"],
  "entities": ["pc01"],
  "schema_fingerprint": "<resolved-context-fingerprint>"
}
```

4.2 Responsibilities

4.2.1 Preserve Original Query

Never lose the exact original question. You need it later for:

- LLM context
- Debugging
- Evaluation
- Logging

4.2.2 Normalize

Normalization helps retrieval but must not claim semantic equivalence that is not established.

User Term	Normalized
“apps”	“applications”
“machines”	“computers”

Table 1: Normalization Examples

4.2.3 Extract Obvious Entities

Separate schema-like terms from data-like values:

Category	Examples	Purpose
Schema-like	`application`, `computer`, `running on`	Retrieval targets
Data-like	`pc01`	Filter values

Table 2: Entity Classification

5 Module 4 — Version-Scoped Schema Retriever

This module is responsible **only** for finding possible schema concepts relevant to the user's question within the selected `OksContext`. It does **not** decide legality and it must never search unfiltered documents from other contexts.

5.1 Retrieval Input / Output

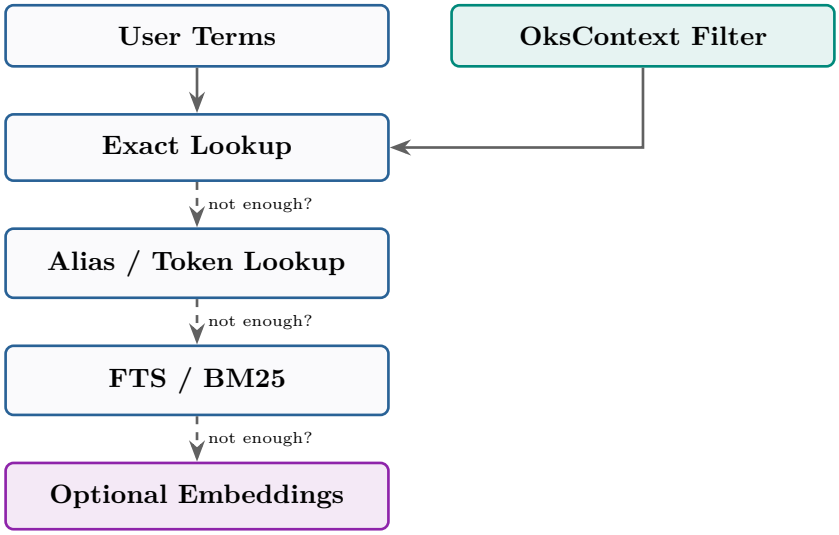
Input: normalized terms plus the resolved schema fingerprint:

```
query = "applications running on computer pc01"
schema_fingerprint = "<resolved-context-fingerprint>"
```

Output:


```
[
  {
    "name": "Application",
    "kind": "class",
    "score": 0.92,
    "source": "exact_name",
    "schema_fingerprint": "<resolved-context-fingerprint>"
  },
  {
    "name": "Computer",
    "kind": "class",
    "score": 0.88,
    "source": "token_match",
    "schema_fingerprint": "<resolved-context-fingerprint>"
  },
  {
    "name": "RunsOn",
    "kind": "relationship",
    "score": 0.75,
    "source": "fts_bm25",
    "schema_fingerprint": "<resolved-context-fingerprint>"
  }
]
```

5.2 Retrieval Order (Cascading)



Rule: Do not make embeddings the first step.

Version-isolation rule: every stage applies the same schema fingerprint filter. A result from another release, revision, configuration, or run is invalid even if its name is a stronger textual match.

5.3 Retrieval Index Structure

Since OKS is the authority, the retrieval index is only a **derived index**. Build one searchable document per class:

```
{
  "schema_fingerprint": "<resolved-context-fingerprint>",
  "git_revision": "<resolved-revision>",
  "class_name": "Application",
  "tokens": ["application", "app"],
  "description": "...",
  "relationships": ["RunsOn", "BackupHosts"],
  "relationship_targets": ["Computer"],
  "attributes": ["Name", "Parameters", "Logging"]
}
```

Listing 1: Class Search Document Example

The index may be physically partitioned by context or stored in one table with a mandatory `schema_fingerprint` field. The required property is logical isolation:

```
search(
    query=user_terms,
    schema_fingerprint=oks_context.schema_fingerprint,
    top_k=10,
)
```

The initial implementation uses *version → actual schema*. It does not introduce base-schema plus delta reconstruction; that optimization is deferred until measurement proves it necessary.

5.4 Why Class-Level Documents?

Do not split arbitrary XML into chunks. One coherent, context-keyed retrieval unit preserves semantic structure:

Class

- superclass
- attributes
- relationships
- description

6 Module 5 — Version-Scoped OKS Schema Access Layer

This is the most important architectural distinction. The retriever says: "I think `Application` and `Computer` are relevant in this context." The schema access layer asks the version-bound OKS kernel: "What **exactly** are `Application` and `Computer` in this `OksContext`?"

6.1 Source

OksSchemaProvider(oks_context)

Version-bound `OksKernel`

Not LLM. Not embedding. Not an unscoped global schema.

6.2 Required Operations

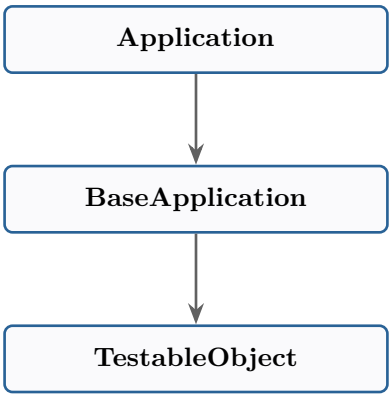
Conceptually equivalent to:

```
provider = OksSchemaProvider(oks_context)

get_class(name)
class_exists(name)
get_superclasses(class_name)
get_attributes(class_name)
get_attribute(class_name, attribute_name)
get_relationships(class_name)
get_relationship(class_name, relationship_name)
get_relationship_target(class_name, relationship_name)
```

The provider is permanently associated with one immutable `OksContext`. An unqualified call such as `schema_provider.get_class("Application")` is forbidden unless the provider was already constructed with that context.

6.3 Effective Members (Inheritance Resolution)

Suppose the inheritance chain:



Then `get_effective_members("Application")` must resolve:

Class	Direct	Inherited
Application	Name	—
BaseApplication	Parameters, Logging	—
TestableObject	UID	—
Effective	Name	Parameters, Logging, UID

Table 3: Effective Member Resolution

6.4 Relationship Resolution

For `Application.RunsOn`, return:

```
{
  "source": "Application",
  "relationship": "RunsOn",
  "target": "Computer"
}
```

This enables path construction: `Application` → `RunsOn` → `Computer`

Every returned class and relationship is tagged internally with the context's schema fingerprint so it cannot be combined accidentally with objects loaded from another revision.

7 Module 6 — Schema Context Builder

This module takes version-filtered retrieval candidates and the authoritative schema from the same `OksContext`, then creates the exact schema context the LLM needs.

7.1 Example

User: "Which applications are running on computer pc01?"

Retriever: `Application`, `Computer`, `RunsOn`

Schema provider verifies:

- `Application.RunsOn` → `Computer`
- `Computer.Name : string`

Context builder produces:

```
RELEVANT SCHEMA

OKS CONTEXT
  Git revision: <resolved-revision>
  Schema fingerprint: <resolved-context-fingerprint>
  Configuration revision: <resolved-configuration>

The definitions below are authoritative for this request.
Do not assume that members from another context exist.

CLASS Application
  Inherited from: BaseApplication
  Relevant relationships:
    RunsOn -> Computer
  Relevant attributes:
    Name : string
    ...

CLASS Computer
  Relevant attributes:
    Name : string

VALID RELATIONSHIP PATH:
  Application -> RunsOn -> Computer
```

7.2 Context Expansion Rules

1. **Expand only what is needed.** If `Application` requires `BaseApplication` (because `RunsOn` is inherited), include it. Do not include all 633 classes.
2. **One-hop relationship expansion.**

   If `Application.RunsOn` → `Computer`, retrieve `Computer`. Only expand further if the query requires it.
3. **Context pruning.** If `Application` has 30 attributes but the question only concerns `RunsOn` and `Name`, do not send unrelated attributes. This reduces LLM confusion.
4. **Context consistency.** Candidate classes, inherited members, relationships, targets, value types, and ranges must all be loaded from the same `OksContext`. Mixed-context expansion is a correctness failure.

8 Module 7 — Query Prompt Builder

Construct the LLM request containing six essential components. The prompt builder receives the same immutable `OksContext` used by retrieval and schema access.

8.1 Component A: Instructions

```
You translate natural language into the provided query AST.
You must only use classes, attributes and relationships present
in the supplied schema context.
Only the supplied OksContext is authoritative for this request.
Do not use members remembered from other releases or revisions.
Do not invent schema members.
Return JSON only.
```

8.2 Component B: Query AST Schema

```
Query:
  root_class
  filters
  projections
  traversals
  ordering
  limit
```

8.3 Component C: OksContext Metadata

The prompt includes the resolved Git revision, configuration revision, schema identifier, and schema fingerprint. These values constrain the model but remain outside the semantic query AST.

8.4 Component D: Schema Context

Generated by Module 6 (Section 7).

8.5 Component E: Examples

Examples teach AST structure rather than unverified schema vocabulary. If an example contains concrete classes or relationships, it must be tagged as compatible with the current schema fingerprint before it can be selected.

```
// Question: Which applications have name App1?
{
  "root_class": "Application",
  "filters": [
    {
      "path": ["Name"],
      "operator": "==",
      "value": "App1"
    }
  ],
  "projection": ["Name"]
}
```

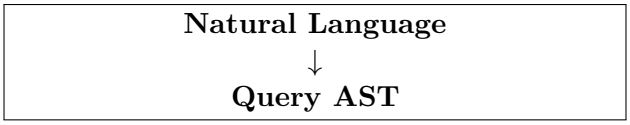
Listing 2: Example AST

8.6 Component F: User Question

The exact original question, preserved from Module 3.

9 Module 8 — LLM Query Generator

This is the only major AI reasoning component. Its task is version-scoped:



Not: Natural Language → Raw OKSQuery.

The generator receives a `QueryRequest` containing the `OksContext` and the prompt inputs. The version is not copied into the semantic AST; it remains attached through the surrounding request.

9.1 Example Output

User: "Which applications are running on pc01?"

LLM generates:

```
{
  "root_class": "Application",
  "filters": [
    {
      "path": ["RunsOn", "Name"],
      "operator": "==",
      "value": "pc01"
    }
  ],
  "projection": ["Name"]
}
```

Listing 3: Generated Query AST

The exact AST structure is defined by the system, not invented by the LLM.

The generated AST is meaningful only together with the request's `OksContext`. Reusing it under another context requires a fresh validation and may produce a different result.

10 Why AST Instead of Raw OKSQuery?

The AST enables inspection of each part before compilation:

- Root class?
- Filter?
- Field?
- Relationship?
- Operator?
- Value?

The complete request is represented as:

```
QueryRequest(
    oks_context=resolved_context,
    ast=query_ast,
)
```

This separates query semantics from the exact configuration and schema against which those semantics are interpreted.

If the LLM directly outputs:

```
Application(RunsOn.Name == "pc01")
```

you must parse the LLM's syntax again. AST eliminates that unnecessary ambiguity.

11 Module 9 — AST Normalizer

LLM output may be structurally valid but inconsistent. The normalizer enforces canonical AST form while preserving the request's `OksContext` unchanged.

11.1 Responsibilities

11.1.1 Normalize Operators

LLM Output	Canonical
`equals`, `=`, `is equal to`	`==`

Table 4: Operator Normalization

11.1.2 Normalize Paths

These should become one canonical representation:

```
RunsOn.Name    ->    ["RunsOn", "Name"]
```

11.1.3 Normalize Values

```
10     // must remain numeric, not "10"
```

11.1.4 Validate AST Structure

Use Pydantic or equivalent. Malformed AST:

```
{"filters": "hello"}   // INVALID: filters must be an array
```

should fail before schema validation.

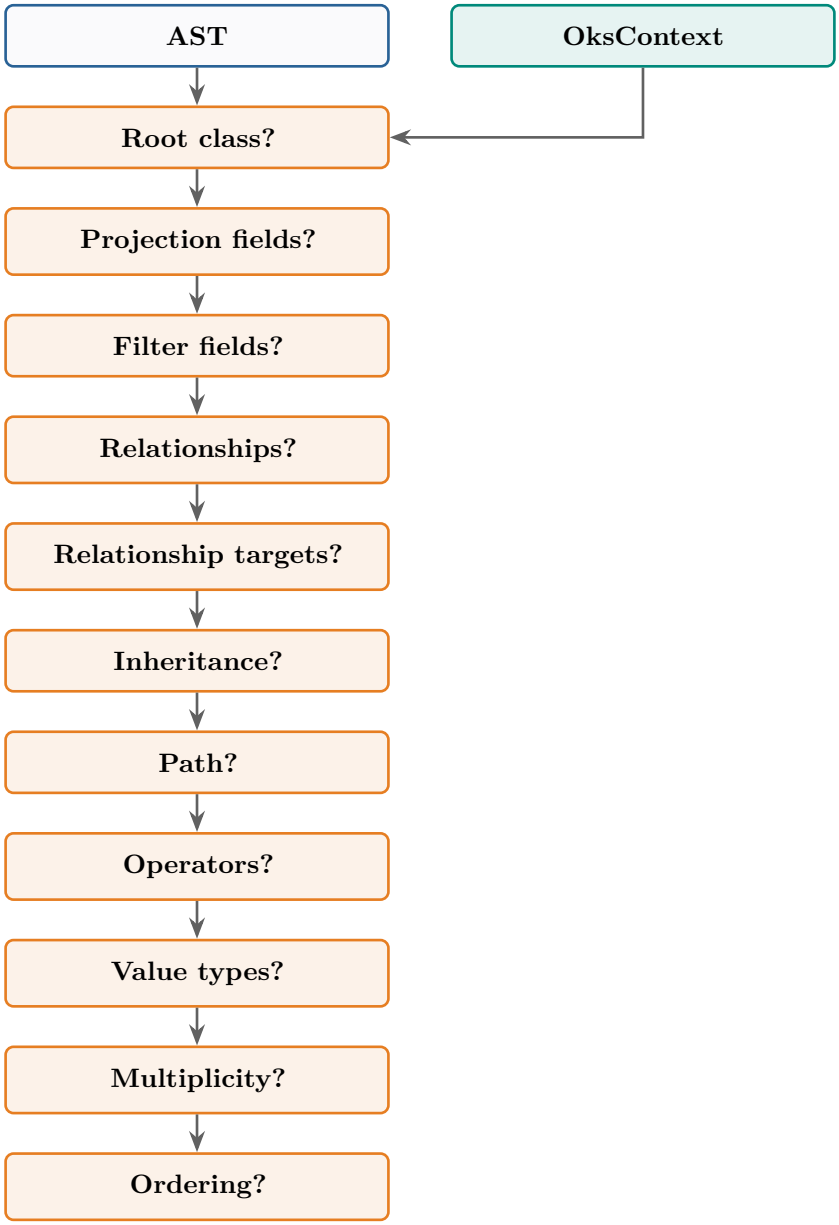
12 Module 10 — Version-Aware Deterministic Schema/AST Validator

This is the hard correctness boundary. It uses:

AST + the request's OksContext → VALID or **INVALID + structured errors**

The validator never asks whether a member exists somewhere in TDAQ. It asks whether the member exists on the relevant class in this exact schema fingerprint.

12.1 Validation Pipeline



12.2 Example: Invalid Field

LLM generates: `Application.RunOn`

Validator asks the bound provider whether `Application.RunOn` exists in the exact request context.

OKS says: **NO**

```
INVALID_FIELD
  Schema fingerprint: <resolved-context-fingerprint>
  Requested: RunOn
  Possible:  RunsOn
```

12.3 Same AST, Different Context

The same semantic AST can have different validation results:

Context	Member	Result
Context A	`Application.RunsOn`	Invalid if absent
Context B	`Application.RunsOn`	Valid if present

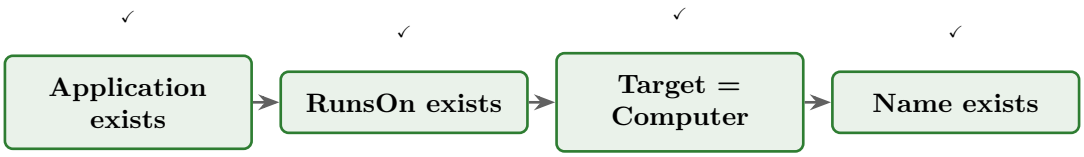
Table 5: Validation is determined by the resolved schema context

This is expected behaviour, not inconsistency: the AST describes query semantics, while the context determines which schema makes those semantics legal.

12.4 Relationship Path Validation

Suppose AST contains: `Application` → `RunsOn` → `Computer` → `Name`

Validator checks each step against the same context fingerprint:



Only if every step is valid can the path be accepted.

12.5 Type Validation

Suppose schema defines: `Timeout : u32`

LLM generates: `Timeout == "hello"`

Schema validator knows: `u32` ≠ `string`

```
INVALID_VALUE_TYPE
  Field: Timeout
  Expected: u32
  Received: string
```

This is something embeddings/RAG cannot reliably guarantee.

12.6 Operator Validation

The validator knows which operators are supported per type:

Type	Allowed Operators
Numeric	`==`, `!=`, `<`, `>`, `<=`, `>=`
String	`==`, `!=`, `contains`, `starts_with`
Boolean	`==`, `!=`

Table 6: Operator Type Restrictions

The exact allowed operators come from OKS query semantics, not from the LLM.

13 Module 11 — Version-Aware Repair Engine

If validation fails, do not necessarily terminate. Send focused feedback to the LLM together with the same `OksContext`. Repair is not allowed to borrow members or examples from another schema fingerprint.

13.1 Example Repair Cycle

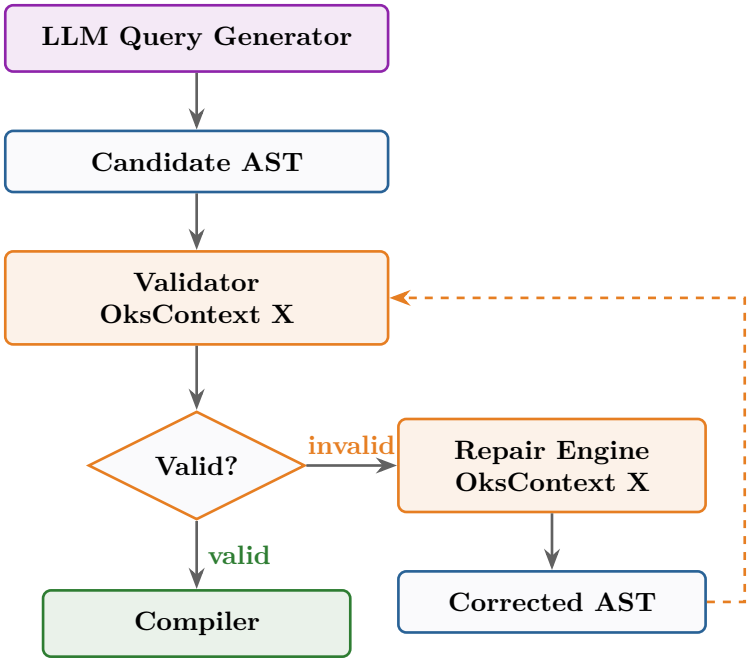
Error: `RunOn` does not exist on `Application`.

Feedback sent to LLM:

```
Your AST contains invalid relationship ‘RunOn‘.

Schema fingerprint: <resolved-context-fingerprint>
Only this context is authoritative.

For Application, valid relationships include:
  - RunsOn -> Computer
  - BackupHosts -> Computer

Correct the AST.
```

LLM produces: `RunsOn`
Then: Validate again.

13.2 Repair Loop Diagram



Hard retry limit: `max_repair_attempts = 2`

14 Module 12 — OKSQuery Compiler

After validation: **VALID QueryRequest → AST → OKSQuery compiler**

This is deterministic code. Compilation retains the same context for grammar selection, execution, logging, and provenance; it never resolves or substitutes a different configuration.

14.1 Example

AST:

```
{
  "root": "Application",
```

```
    "filter": "RunsOn.Name == 'pc01'"
  }
```

Becomes: OKSQuery syntax according to the actual OKS query grammar, accompanied by the schema fingerprint used to validate it.

14.2 Important Rule

The compiler should **never** ask the LLM how to write the syntax.

Component	Responsibility
LLM	*What* the user wants
Compiler	*How* OKSQuery expresses it
OksContext	*Which exact schema/configuration gives it meaning*

15 Recommended Package Structure

The implementation should make context resolution an explicit package, not a hidden option inside retrieval or validation:

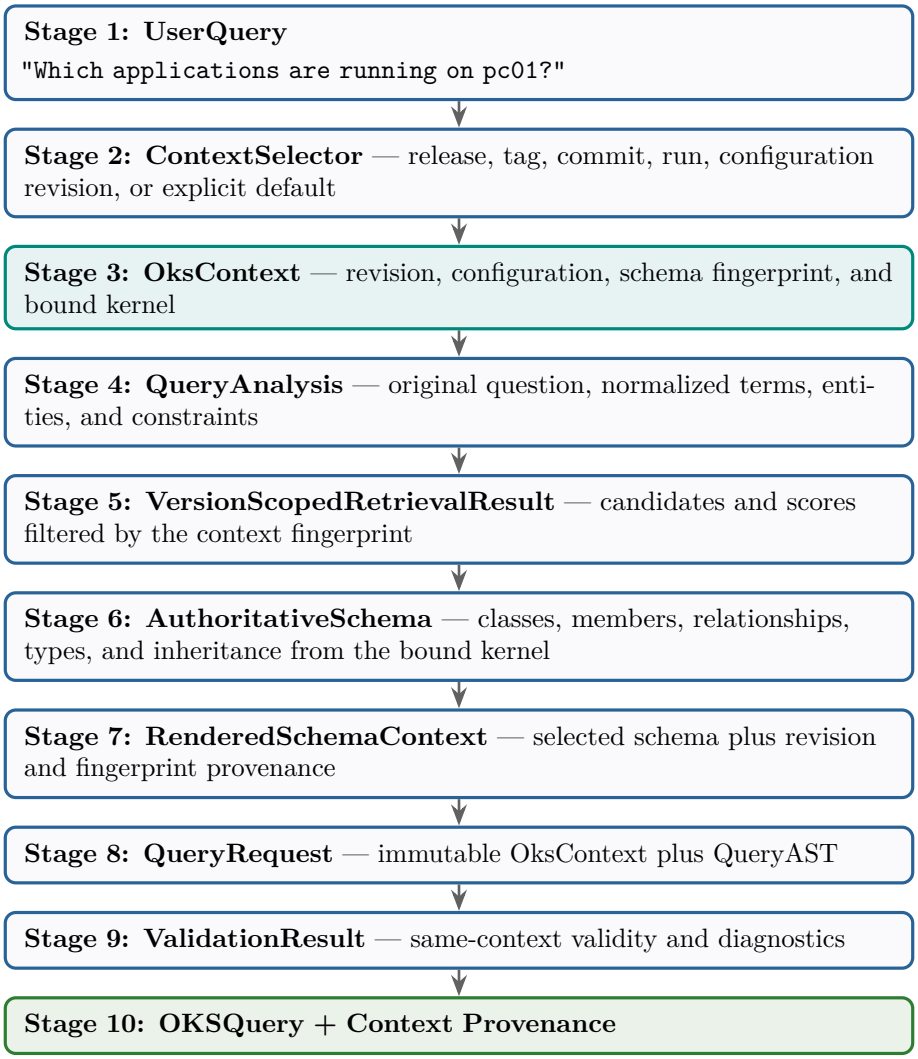
```
src/oksquery/
  context/
    oks_context.py
    version_resolver.py
    run_resolver.py
  retrieval/
    schema_retriever.py
    schema_index.py
    exact_lookup.py
    alias_lookup.py
    fts.py
    embeddings.py          # optional
  schema/
    oks_schema_provider.py
    inheritance.py
    relationship.py
  context_builder/
    builder.py
    renderer.py
  translation/
    prompt_builder.py
    generator.py
    repair.py
  ast/
    models.py
    normalizer.py
    validator.py
    compiler.py
  execution/
    executor.py
```

The `OksContext` instance is passed into these components rather than reconstructed independently inside them.

16 Final Architecture by Responsibility

Module	Main Question It Answers	Deterministic
Version / Run Resolver	Which release, revision, run, or configuration applies?	Yes
OksContext Builder	Which immutable kernel and schema fingerprint bind this request?	Yes
Query Preprocessor	What terms did the user use?	Mostly
Schema Retriever	What concepts might be relevant in this context only?	Yes/mostly
OKS Schema Access	What actually exists in the selected schema?	Yes
Context Builder	What version-scoped schema should the LLM see?	Yes
Prompt Builder	How should we present it?	Yes
LLM Generator	What query does the user mean?	No
AST Normalizer	Is the output structurally canonical?	Yes
AST Validator	Is the query legal in this exact OksContext?	Yes
Repair Engine	Can it be corrected using only same-context evidence?	LLM-assisted
Compiler	How is the validated AST expressed and traced to its context?	Yes

17 Information Flow Through the System



18 What Is Stored Permanently?

You do not need to permanently duplicate every complete schema in another database. The authoritative schema remains the version-bound OKS configuration loaded by `OksKernel`.

18.1 Permanent Storage

- OKS itself
- Version-filtered retrieval documents keyed by schema fingerprint
- Aliases keyed by schema fingerprint where vocabulary differs
- Prompts
- Structural examples or examples tagged with compatible contexts
- Evaluation data including the resolved context and fingerprint
- Optional session/cache metadata for already loaded contexts

18.2 Dynamically Obtained

From the `OksKernel` bound to each request's `OksContext`:

- Exact class definitions
- Attributes
- Relationships
- Inheritance
- Schema identifier and schema fingerprint inputs

The retrieval index can be regenerated from OKS whenever the configuration/schema changes. A base-schema plus version-delta storage model is deliberately deferred: correctness and direct correspondence to the actual schema take priority over deduplication.

19 Recommended Retrieval Index

For a 633-class scale, use one `ClassSearchDocument` per class and per schema fingerprint:

```
{
  "schema_identifier": "<schema-id>",
  "schema_fingerprint": "<schema-fingerprint>",
  "git_revision": "<git-revision>",
  "configuration_revision": "<configuration-revision>",
  "class_name": "Application",
  "tokens": [
    "application",
    "app"
  ],
  "description": "...",
  "relationships": [
    "RunsOn",
    "BackupHosts"
  ],
  "relationship_targets": [
    "Computer"
  ],
  "attributes": [
    "Name",
    "Parameters",
    "Logging"
  ]
}
```

Listing 4: Class Search Document

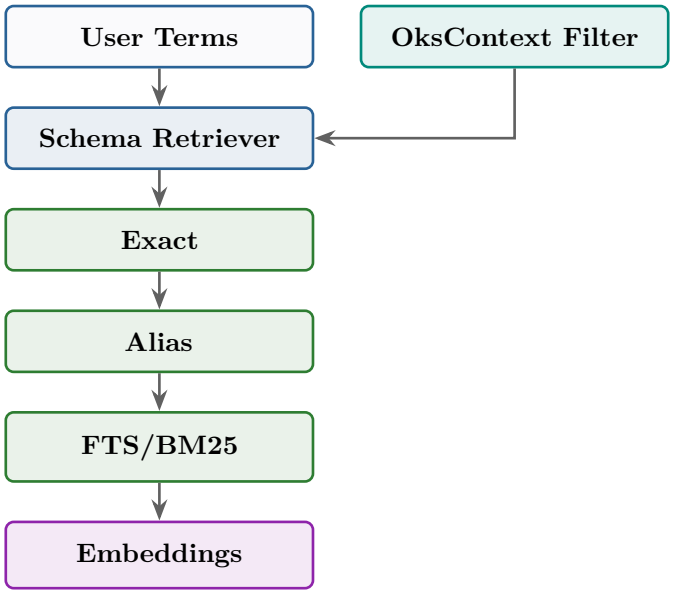
Retrieval always includes the context filter:

```
search(
    query="applications running on computer",
    schema_fingerprint=oks_context.schema_fingerprint,
)
```

This is sufficient to perform: **exact** → **alias** → **BM25** before considering embeddings, without exposing documents from another schema.

20 Where RAG Belongs

If you use RAG, put it **only** here:



Never:

```
Cross-version RAG -> "This field probably exists" -> OKSQuery    //
    WRONG
```

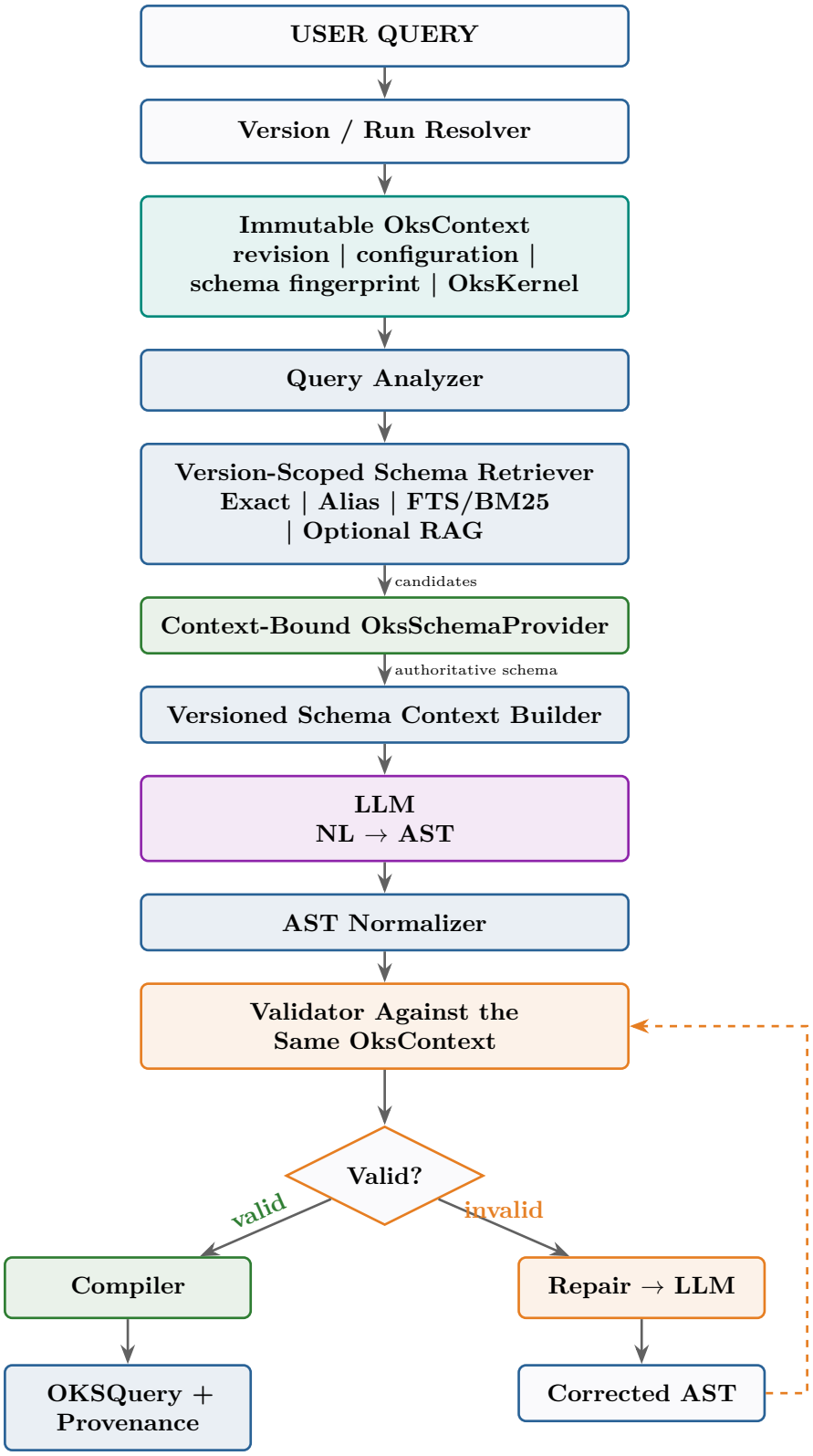
Instead:

```
Context-filtered RAG -> candidate -> context-bound OksKernel ->
    authoritative schema -> LLM
```

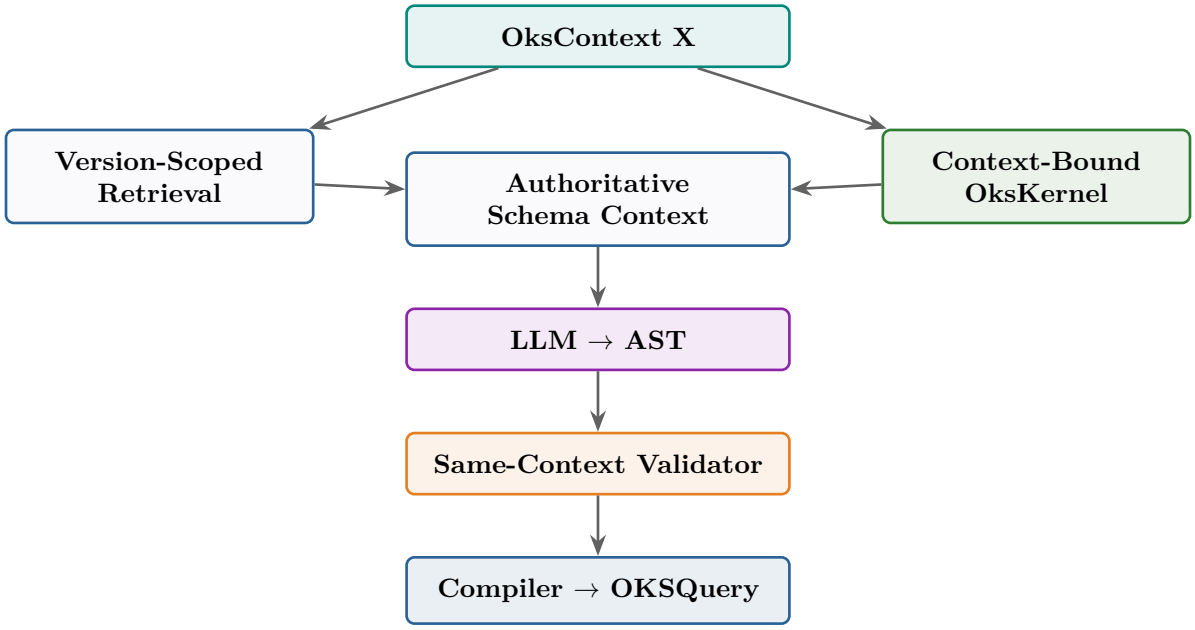
Optional embeddings may rank candidates only inside the selected context. They never establish that a member exists and never bypass the version-bound schema provider.

21 The Final Simplified Architecture

Reduced to essential components:


USER QUERY
Version / Run Resolver
Immutable OksContext
revision | configuration |
schema fingerprint | OksKernel
Query Analyzer
Version-Scoped Schema Retriever
Exact | Alias | FTS/BM25
| Optional RAG
candidates
Context-Bound OksSchemaProvider
authoritative schema
Versioned Schema Context Builder
LLM
NL → AST
AST Normalizer
Validator Against the
Same OksContext
Valid?
valid
invalid
Compiler
Repair → LLM
OKSQuery +
Provenance
Corrected AST

22 The Core Principle



Resolve once, then use the same immutable OksContext for retrieval, schema expansion, prompting, validation, repair, compilation, and execution.